Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int firstZero(int arr[], int low, int high) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == 0 && (mid == 0 || arr[mid - 1] == 1))
        return mid;

    if (arr[mid] == 1)
        return firstZero(arr, mid + 1, high);
    else
        return firstZero(arr, low, mid - 1);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int index = firstZero(arr, 0, n - 1);

    if (index == -1)
        printf("0");
    else
        printf("%d", n - index);

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔

	Input	Expected	Got	
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int countOccurrences(int nums[], int left, int right, int value) {
    int count = 0;

    for (int i = left; i <= right; i++) {
        if (nums[i] == value)
            count++;
    }

    return count;
}

int majorityElement(int nums[], int left, int right) {

    if (left == right)
        return nums[left];


    int mid = left + (right - left) / 2;


    int leftMajority = majorityElement(nums, left, mid);
    int rightMajority = majorityElement(nums, mid + 1, right);


    if (leftMajority == rightMajority)
        return leftMajority;


    int leftCount = countOccurrences(nums, left, right, leftMajority);
    int rightCount = countOccurrences(nums, left, right, rightMajority);

    if (leftCount > rightCount)
        return leftMajority;
    else
        return rightMajority;
```

```
38 }
39
40 int main() {
41     int n;
42     scanf("%d", &n);
43
44     int nums[n];
45
46     for (int i = 0; i < n; i++)
47         scanf("%d", &nums[i]);
48
49     int result = majorityElement(nums, 0, n - 1);
50
51     printf("%d\n", result);
52
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findFloor(int arr[], int low, int high, int x) {
    if (low > high) {
        return -1;
    }

    if (x >= arr[high]) {
        return arr[high];
    }

    int mid = low + (high - low) / 2;

    if (arr[mid] == x) {
        return arr[mid];
    }

    if (arr[mid] > x) {
        return findFloor(arr, low, mid - 1, x);
    }

    int floorInRight = findFloor(arr, mid + 1, high, x);
    return (floorInRight != -1) ? floorInRight : arr[mid];
}

int main() {
    int n, x;

    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);

    int floorValue = findFloor(arr, 0, n - 1, x);

    printf("%d\n", floorValue);

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>


int recursive_binary_search(int arr[], int low, int high, int target) {
    if (low > high) {
        return 0;
    }

    int mid = low + (high - low) / 2;

    if (arr[mid] == target) {
        return 1;
    } else if (arr[mid] > target) {
        return recursive_binary_search(arr, low, mid - 1, target);
    } else {
        return recursive_binary_search(arr, mid + 1, high, target);
    }
}

int find_pair_recursive(int arr[], int low, int high, int x, int *elem1, int *elem2) {
    if (low >= high) {
        return 0;
    }

    int current_element = arr[low];
    int complement = x - current_element;

    if (recursive_binary_search(arr, low + 1, high, complement)) {
        *elem1 = current_element;
        *elem2 = complement;
        return 1;
    }

    return find_pair_recursive(arr, low + 1, high, x, elem1, elem2);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    int elem1, elem2;
    int found = find_pair_recursive(arr, 0, n - 1, x, &elem1, &elem2);

    if (found) {
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void quickSort(int a[], int low, int high)
{
    if (low < high)
    {
        int i = low;
        int j = high;
        int pivot = a[low];
        int temp;

        while (i < j)
        {
            while (a[i] <= pivot && i < high)
                i++;

            while (a[j] > pivot)
                j--;

            if (i < j)
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        // Place pivot in its correct position
        temp = a[low];
        a[low] = a[j];
        a[j] = temp;

        // Divide and conquer
        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main()
{
    int n, i;
    int a[100];

    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course